Team IT1E – Technical Design

Year 1 Period 3

Table of Contents:

Table of Contents: *2*

Introduction: *3*

Standards and guidelines: *3*

- **Requirements and design method** 3
- **Hardware standards** 3
- **Software and coding guidelines** 3
- **Project and version control** 3

Project Context *3*

- **Line Following** 4
- **Line Maze** 4
- **Physical Maze** 4

Architecture *5*

- **System Overview** 5
- **High-Level Architecture** 5
- **On-board system** 5
- **External interaction** 6
- **Software Architecture** 6
  - Perception Layer 6
  - Navigation Layer 6
  - Manipulation Layer 11
  - Control Layer 11
  - Safety Layer 11

Hardware *12*

Software *12*

Data storage *13*

Architecture overview *13*

Introduction:

In this project we design and implement an embedded control system for the BattleBot, a 3D-printed Arduino Nano-based robot car for autonomous relay and racing games. The technical design describes the complete hardware and software architecture of our BattleBot (motors, sensors, PCB connections and control software) using the specifications and examples from the module book and “The Ultimate BattleBot Guide” as the main technical references.

Standards and guidelines:

Our BattleBot design follows the module BattleBot standards and the technical guidelines from “The Ultimate BattleBot Guide”, which documents the BattleBot hardware and Arduino usage.

Requirements and design method

- The design includes a clear architecture/block diagram, a complete I/O list, and descriptions of all sensors, actuators and their behaviour on the BattleBot.

Hardware standards

- The BattleBot is built around the Arduino Nano and the supplied PCB, motor driver and sensors, as described in “The Ultimate BattleBot Guide”.

Software and coding guidelines

- Arduino C/C++ code follows the “C++ Style Guide for Arduino projects – Makerguides.com” for naming, formatting and commenting.
- The software uses the standard Arduino structure (setup(), loop()), I/O functions and timing (delay, millis()) as presented in “The Ultimate BattleBot Guide”.

Project and version control

- Git is used to set up and maintain the development environment and to version all BattleBot code and design documents, as required by the “Manage & Control” learning outcome.

Project Context

The project consists of three main segments: line following, a line maze and a physical maze. The segments have the following characteristics in common:

- The robots need to pick up a cone at their starting position and drop it off at their ending position.
- The robots are required to indicate the direction they are going via NeoPixels. The LEDs should have different colors when moving forwards, backwards, turning right and turning left.

Line Following

- The line following robot starts by detecting (using its ultrasonic sensors) the starting flag being raised.
- The robot follows the line after picking up the cone.
- Objects are placed in the path of the robot. The robot must detect them and avoid them, finding its way back to the line and continuing forwards.
- The robot drops off the cone for the second robot, which is going to complete the line maze.

Line Maze

- The robot solving the line maze starts by detecting the cone being placed in front of it by the line following robot.
- The robot enters its line maze solving logic, after picking up the cone.
- The robot follows either the right or the left side, taking the same direction turn first at every intersection.
- Objects are placed on the line. The robot must detect them and treat those branches as dead ends, turning around and going back.
- The robot drops off the cone for the last robot, which is going to complete the physical maze.

Physical Maze

- The robot solving the physical maze by detecting the cone being placed in front of it by the line maze solving robot.
- The robot enters its physical maze solving logic, after picking up the cone.
- The robot follows either the right or left wall, taking the same direction turn at every intersection.

Architecture

System Overview

The system consists of an autonomous robotic car designed to complete three sequential race stages:

1. Line Follower
2. Line Maze
3. Physical Maze

The robot must navigate through each stage without human intervention and reach the finish line as quickly as possible. The maze layout is unknown in advance, so the robot must rely entirely on its sensors and on-board logic. The three robots must work together to pass on an item at “relay” points, where one robot picks up the item, completes its course carrying the item, and drops it off at the relay point to the next robot.

High-Level Architecture

Each robot will have functions for moving forward, left, right, and backwards, along with interacting with the claws on the robot. The functions will pass parameters that specify how much to move forward, how much to turn left or right, etc. These functions will be called based on data processed by the robot’s sensors. The analog brightness or the ultrasound sensors will be constantly read by functions that interpret the information and output it as a clean, calibrated value. The core of the program will use the information from the functions that pull sensor values to algorithmically call the movement and claw functions to guide the robot across the maze the robot has been programmed for. All three robots will have programs that vary based on the maze they are made for. For example, the line following robot will not need the ultrasound sensors, and the physical maze robot will not need the brightness sensors.

On-board system

Responsible for:

- Line Detection
- Wall Detection
- Motion Control
- State Management

External interaction

- Optional Bluetooth module

All critical functions are executed locally on the robot.

Software Architecture

The robot software is organized into functional layers.

Perception Layer

Responsible for reading and processing sensor data.

Functions:

- Read line sensor values
- Read ultrasonic distance

Outputs:

- Line position information
- Distance to obstacles

Navigation Layer

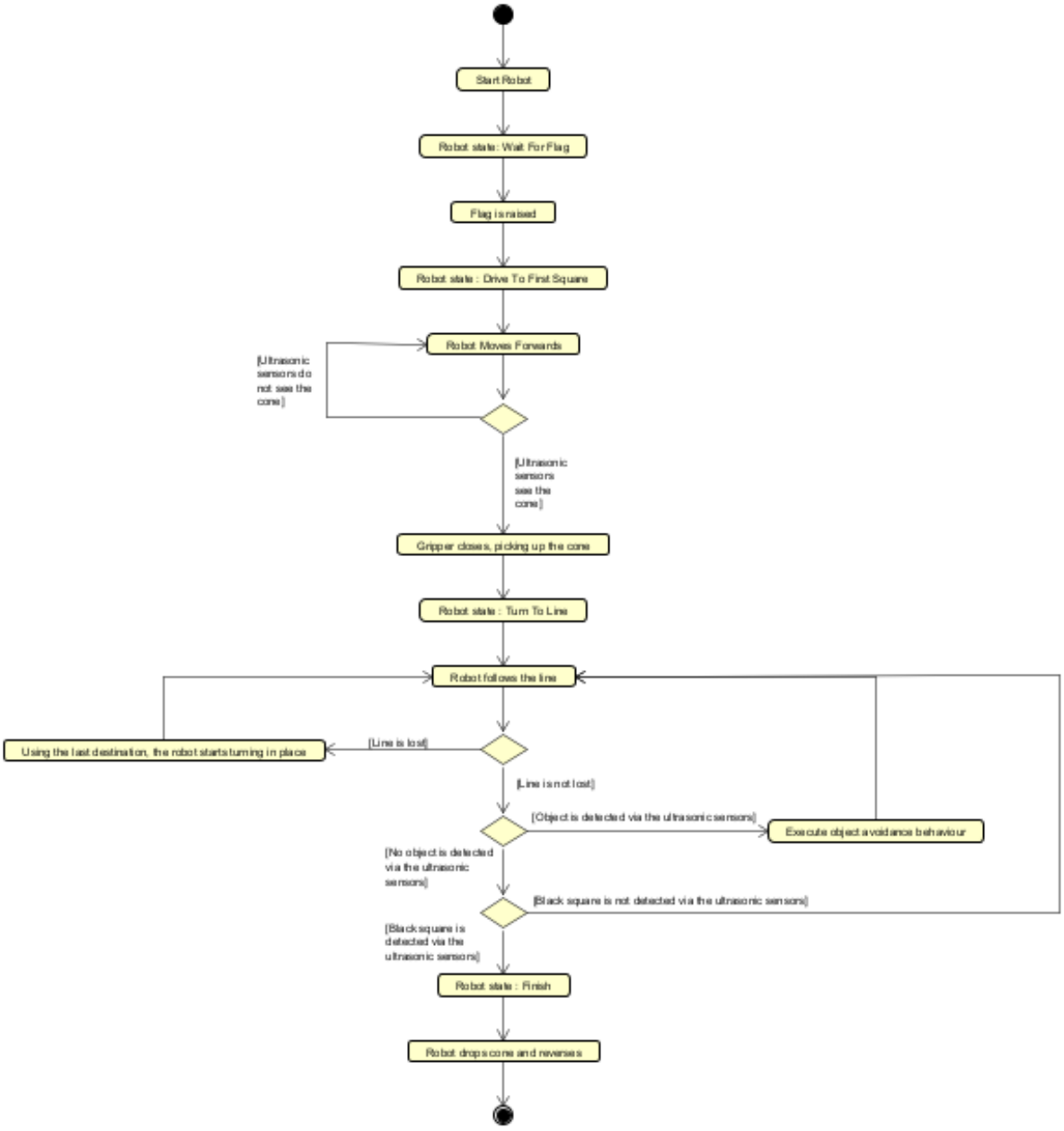
Responsible for deciding how the robot moves depending on its assigned stage.

Each robot has its own navigation logic:

Robot 1 – Line Following

- Follows a single line using the line sensor.
- Continuously adjusts motor speeds to stay on the line.
- Moves toward the first transfer zone.

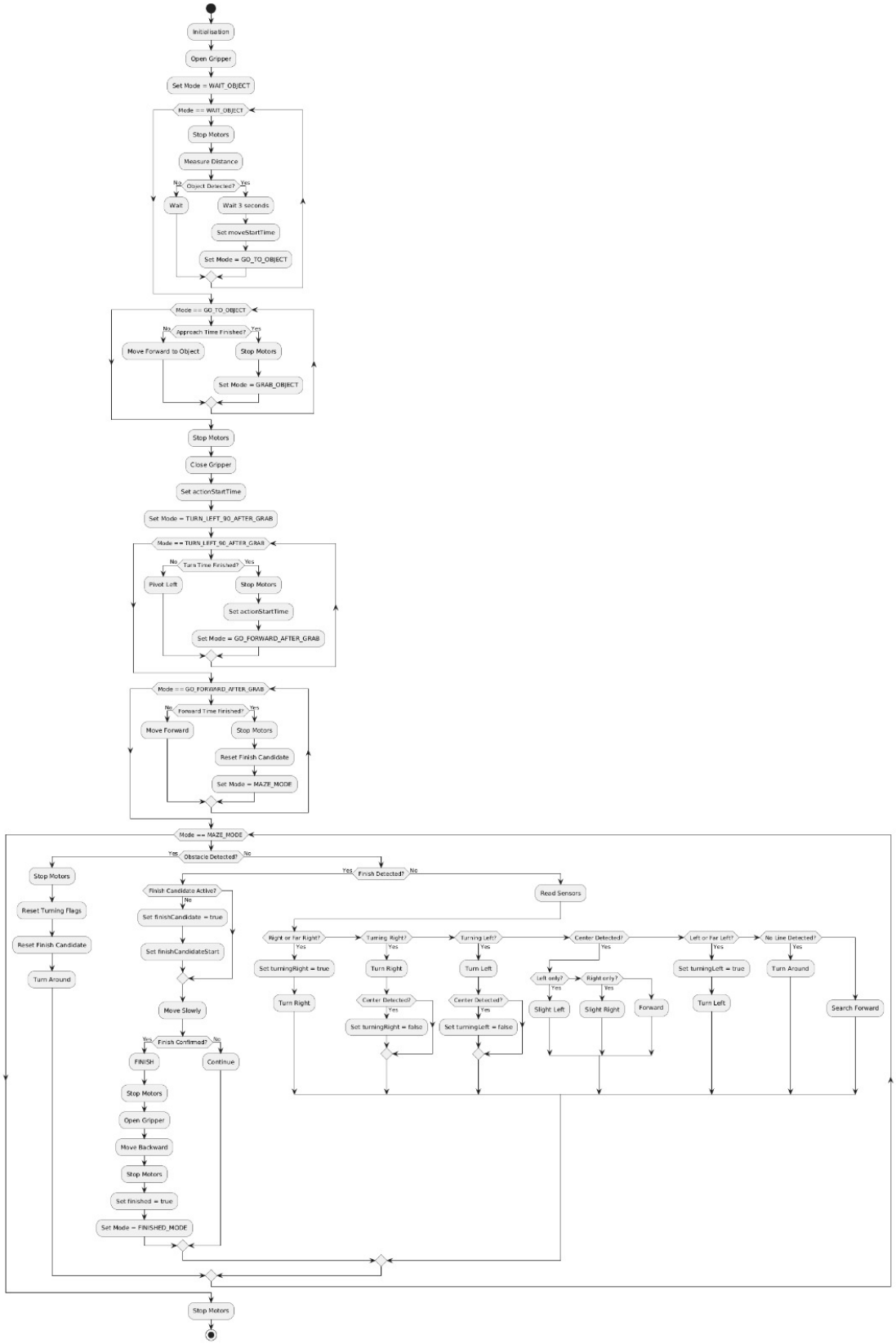
Activity Diagram – Kristian Endrev, Réka Szunyogh-Kocsis:



Robot 2 - Line Maze

- The robot uses the line sensor to detect if it is close to a boundary line.
- If a line is detected under the sensor, the robot corrects its movement to return to the free corridor.
- Chooses direction using a simple maze-solving rule (for example, right-hand rule).
- Moves toward the second transfer zone.

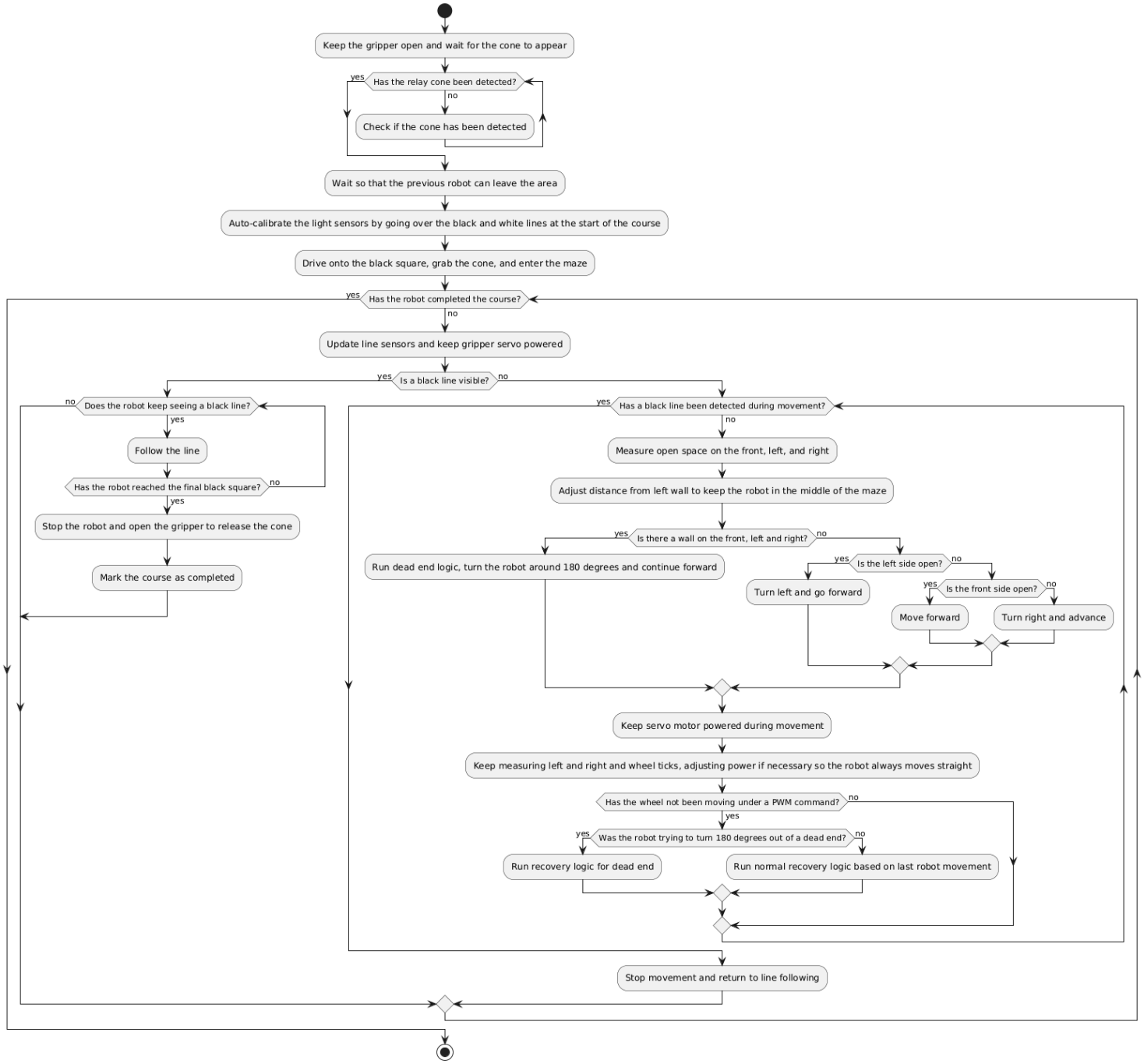
Activity Diagram – Oleksii Khomiak, Mohammadamin Kiani:



Robot 3 - Wall Maze

- The robot uses its ultrasonic sensor(s) to detect if it is close to a wall.
- If a wall is detected, the robot adjusts its movement to return to the optimal position (equal distance between the walls on the left and right).
- Chooses direction using a simple maze-solving rule (for example, right-hand rule).
- Moves towards the finish.

Activity Diagram – JP Moro Machado, Lorenzo Nocivelli:



Manipulation Layer

Responsible for handling the object during the race.

Functions:

- Open and close the gripper
- Pick up the object
- Place the object in transfer zones or at the finish

Control Layer

Responsible for controlling the motors and actuators.

Functions:

- Convert navigation commands into motor signals
- Control left and right motor speeds
- Control the gripper mechanism

Inputs:

- Target speed
- Target direction
- Gripper commands

Outputs:

- Gripper control signal

Safety Layer

Responsible for handling abnormal or unsafe conditions.

Functions:

- Stop the robot if an obstacle is too close
- Stop if the line is lost for too long
- Handle sensor failures

Data Flow

The robot operates in a continuous control loop:

1. Sensors read environment data.
2. Perception layer processes sensor values.

3. Navigation layer decides movement.
4. Control layer sends commands to motors and gripper.
5. Loop repeats continuously.

Hardware

- For each Arduino Nano:
  - Microcontroller: ATmega328P
  - Operating Voltage: 5V
  - Input Voltage (recommended): 7-12V
  - Input Voltage (limits): 6-20V
  - Digital I/O Pins: 14 (of which 6 provide PWM output)
  - Analog Input Pins: 8
  - DC Current per I/O Pin: 20mA
  - Flash Memory: 32KB (of which 2 KB used by bootloader)
  - SRAM: 2 KB
  - EEPROM: 1 KB
  - Clock Speed: 16 MHz
  - Size: 45mm x 18mm
  - Weight: 7 g
- Additional items:
  - Bluetooth module
  - I2C Expansion PCB
  - Jumper cables
  - USB programming cable
  - USB power cable
  - Gripper
  - Motors
  - LEDs
  - 4 neopixels
  - 3 Buttons
  - Line sensor
  - Ultrasonic sensor

Software

- Arduino Legacy IDE (C++)

- Arduino Libraries

Data storage

- A GitHub repository will be used for the team, including all work of the three sub-groups, organized as follows in this example:

Architecture overview

The BattleBot control software is designed using a layered Arduino-based architecture that follows the standard Arduino program structure while supporting multiple navigation tasks. The architecture separates sensor handling, decision logic, and motor control, allowing the robot to navigate different maze types using the same hardware configuration.

The software runs a single continuous program on the Arduino Nano and executes autonomously if the robot is powered.

Architectural layers

Maze control

- Line Following
- Line Maze Navigation
- Physical Maze Navigation

Decision taking and state control

- Conditions and rule-based decisions
- State transitions (forward, backward, turn, stop)